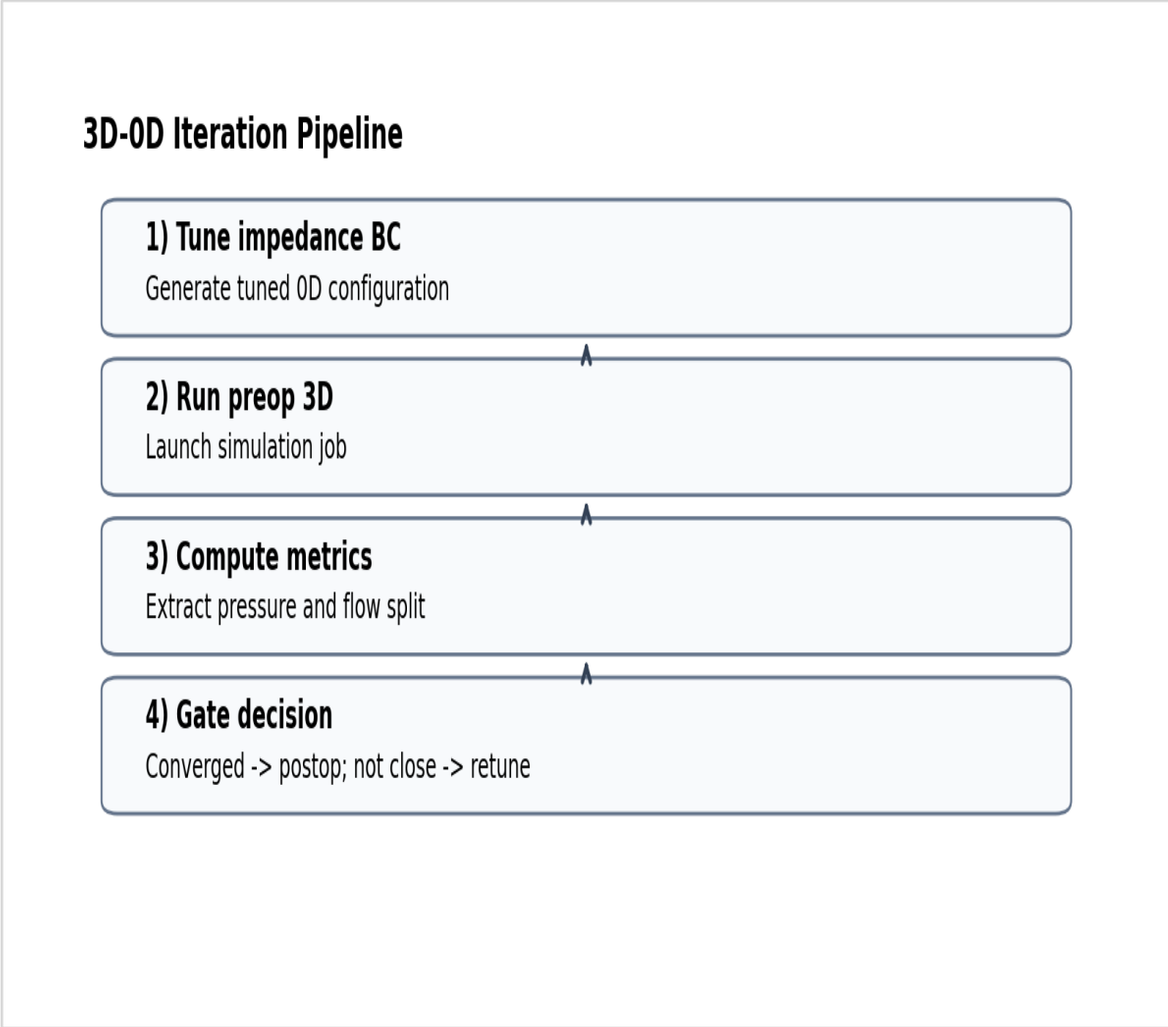
Impedance BC Implementation Review

MASTER-focused check: localized solver impact, BC behavior, and workflow implications



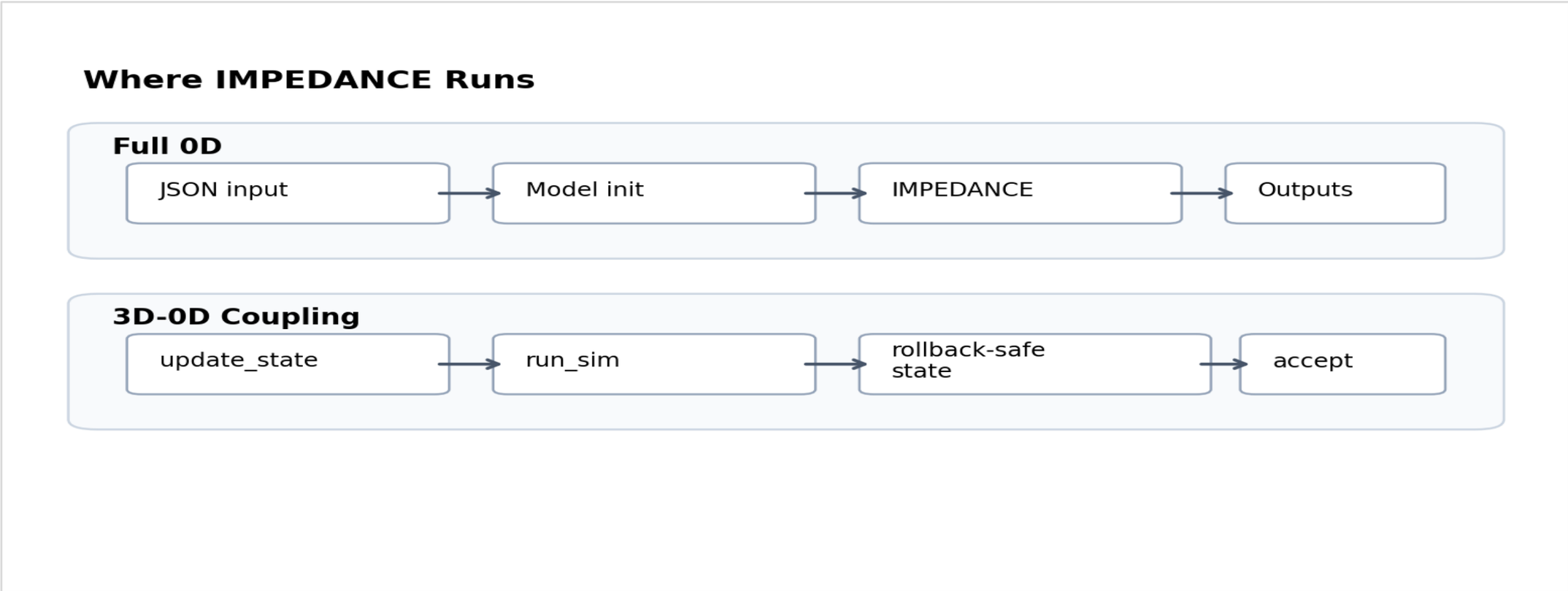
This revision focuses on:

- Structural impact against master
- What changed inside IMPEDANCE BC
- Why minimal support hooks were added
- What this means for 3D-0D and full 0D runs

Top line: no broad solver rewrite was introduced.

Architecture Context

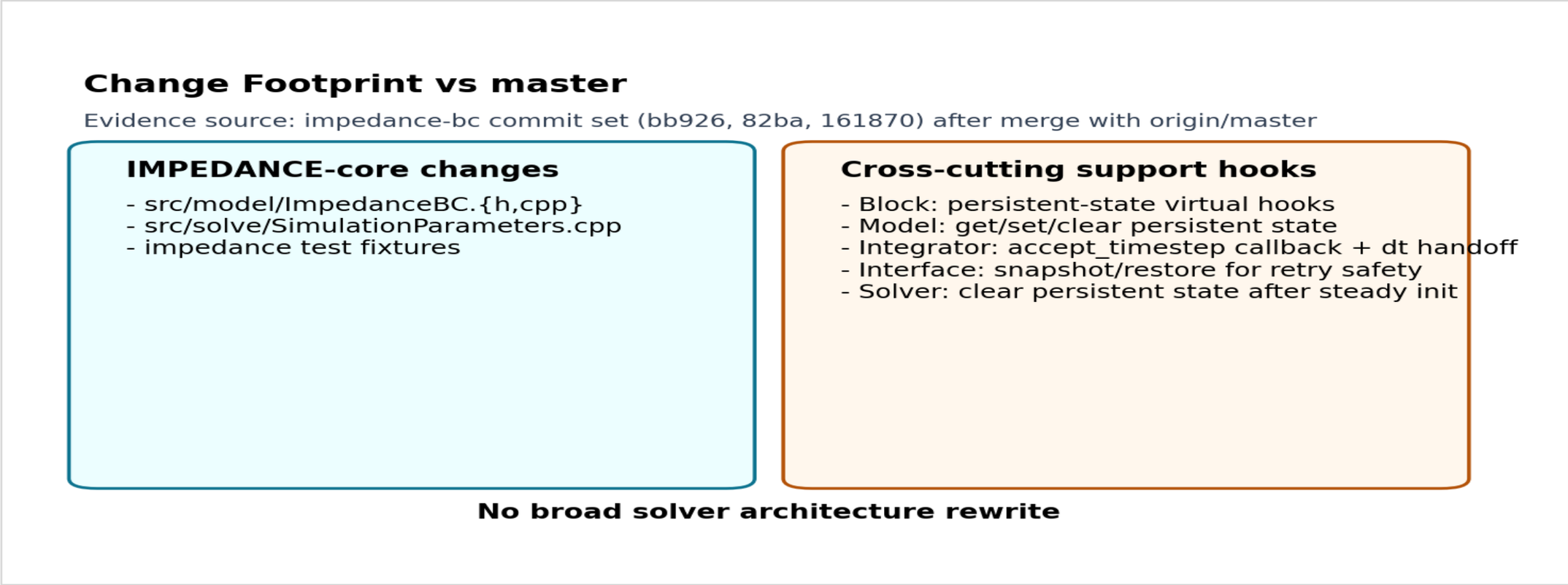
The same IMPEDANCE logic is used in both full 0D and coupled 3D-0D workflows



- Full 0D: standard model execution path
- 3D-0D: retry loop needs rollback-safe internal state
- Shared IMPEDANCE runtime rules across both modes

Structural Footprint vs master

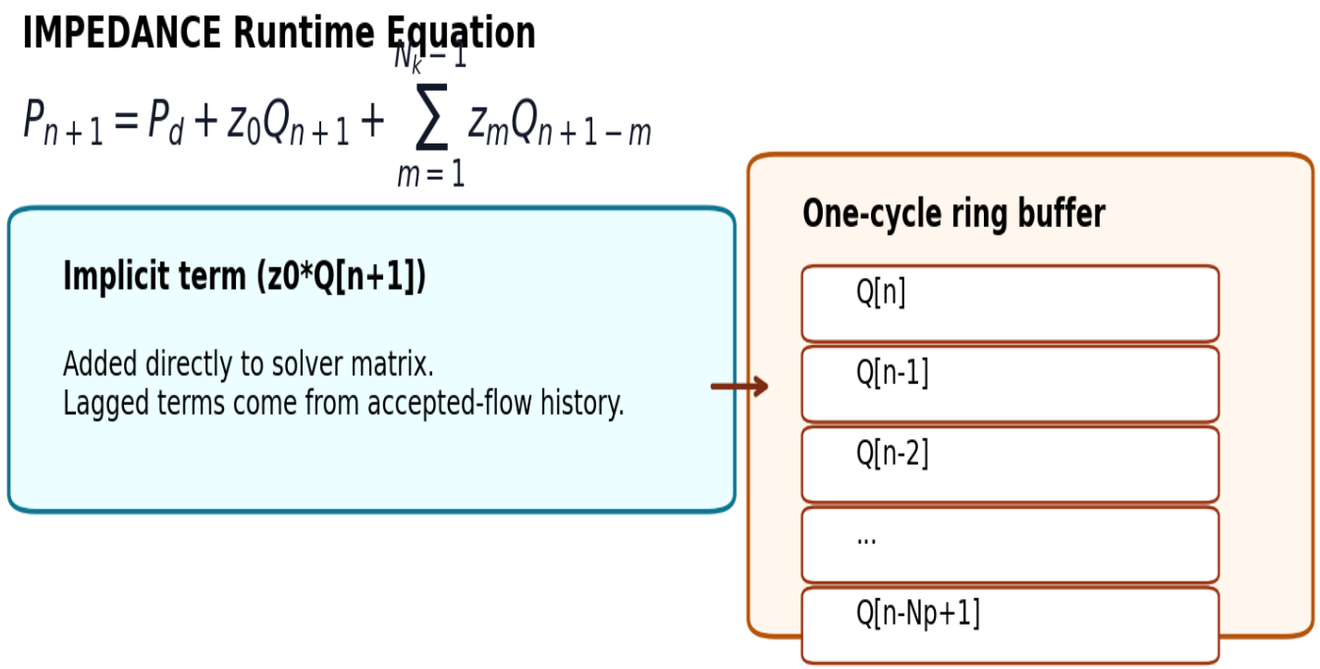
Changes are concentrated in IMPEDANCE logic plus a small set of support hooks



- Core additions stay inside IMPEDANCE BC and its config wiring
- Support hooks are limited to state snapshot/restore and timestep acceptance
- No solver loop redesign or global architecture change

How IMPEDANCE BC Works

Pressure is computed from current flow plus accepted one-cycle flow history

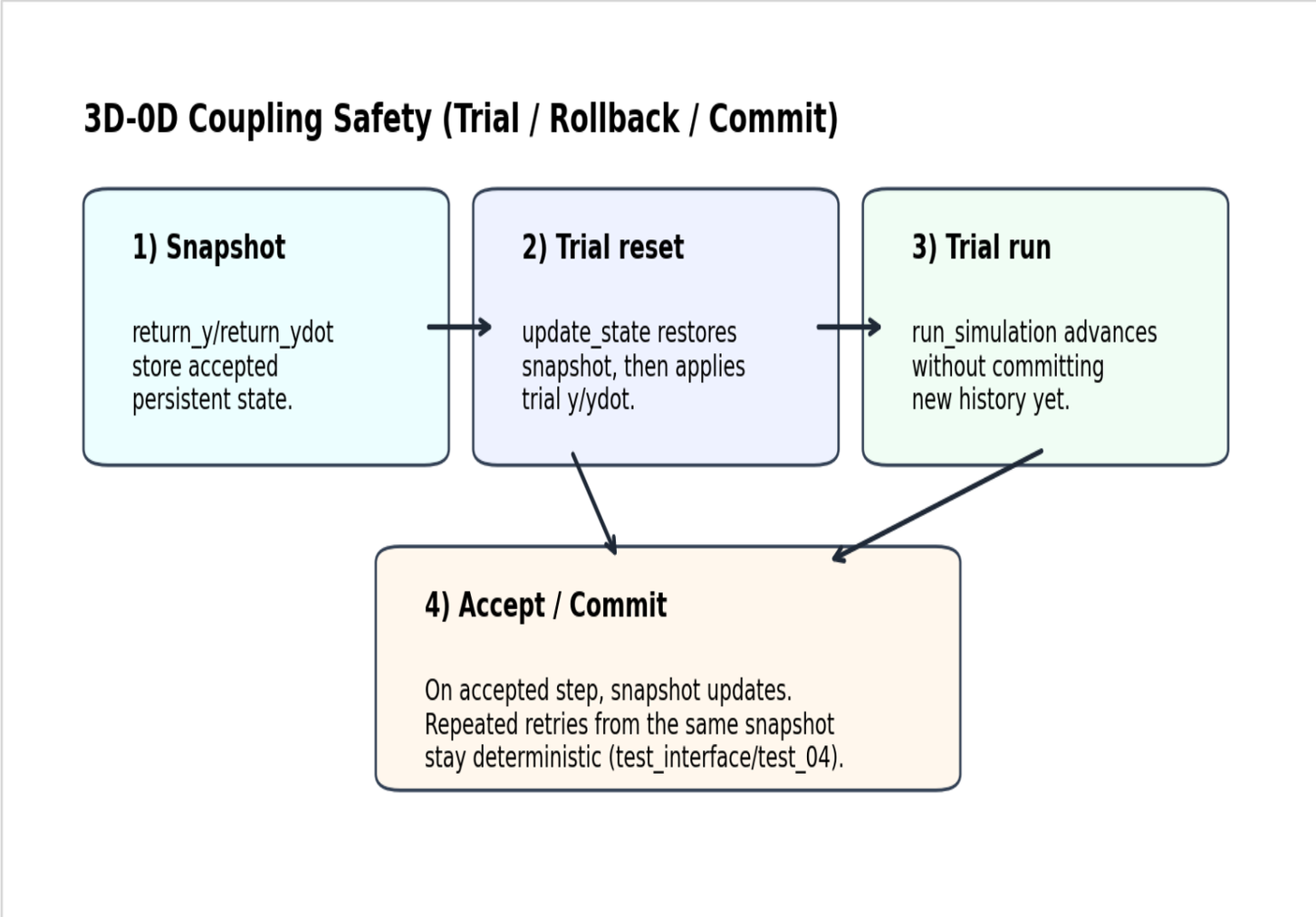


Key runtime behaviors

- Implicit current-flow term enters matrix
- Lagged terms use accepted history only
- Startup follows one-cycle warm-up semantics
- Supports exact or truncated kernel mode

Support Hooks Outside IMPEDANCE

Small cross-cutting additions were made to keep coupling retries safe and deterministic



- Interface snapshots accepted persistent state
- update_state restores snapshot before each trial
- Integrator calls accept_timestep only on accepted steps
- Steady-init path clears dt-dependent persistent memory

Why: preserve deterministic retries and avoid stale-memory carryover.

IMPEDANCE vs RCR

Both remain supported; choice depends on fidelity needs and runtime budget

IMPEDANCE vs RCR (Quick Selection)

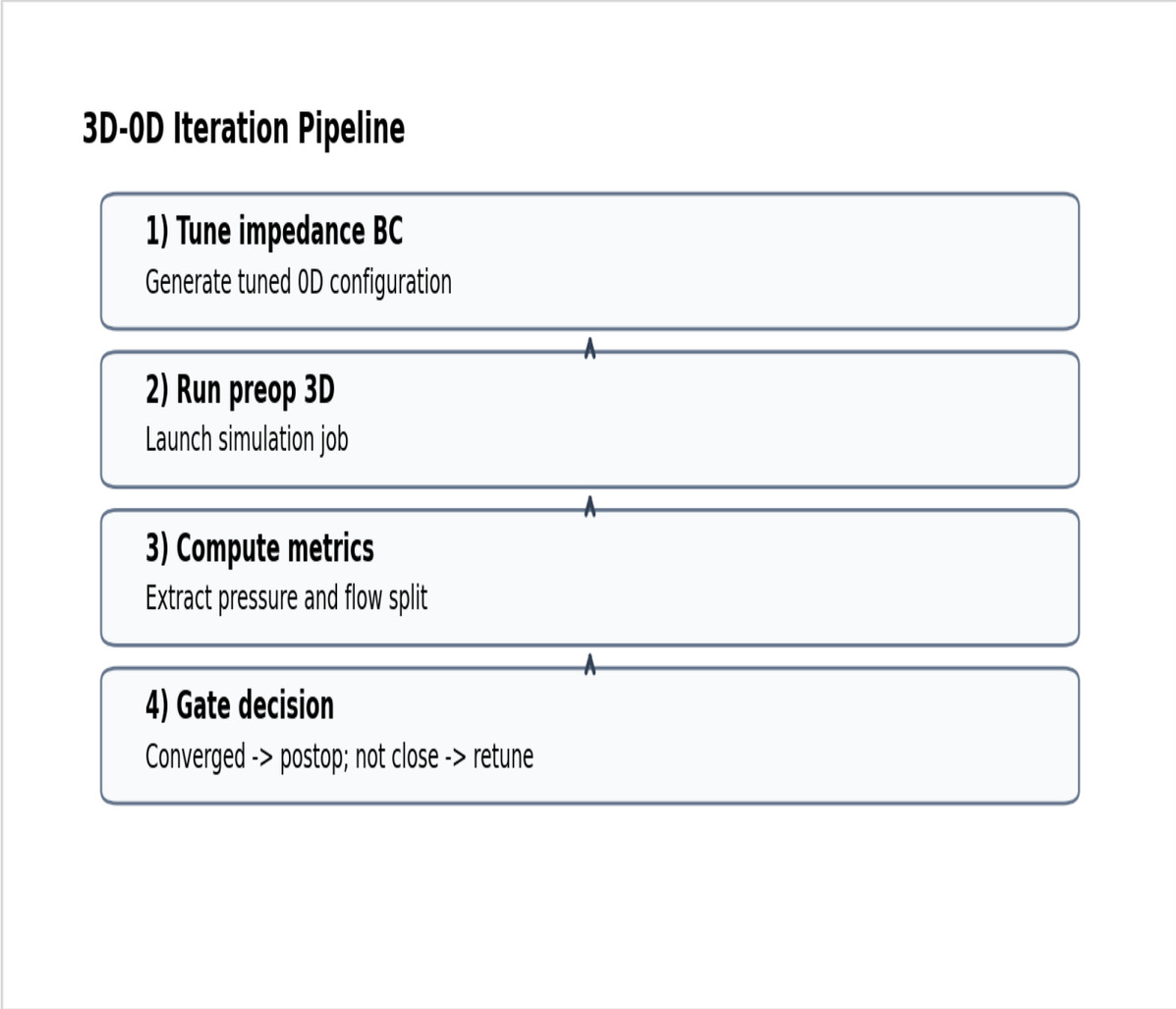
Dimension	IMPEDANCE	RCR
History	One-cycle flow memory	Compact internal state
Behavior	Richer waveform effects	Simpler approximation
Compute cost	Higher	Lower
Best use	Higher-fidelity studies	Fast baseline sweeps

- IMPEDANCE: richer waveform behavior with memory
- RCR: faster baseline model with simpler dynamics
- Select model type based on study objective

Comparison is behavioral and use-case driven.

Practical Implications

Impact summary for coupled 3D-0D tuning loops and standard full 0D execution



3D-0D

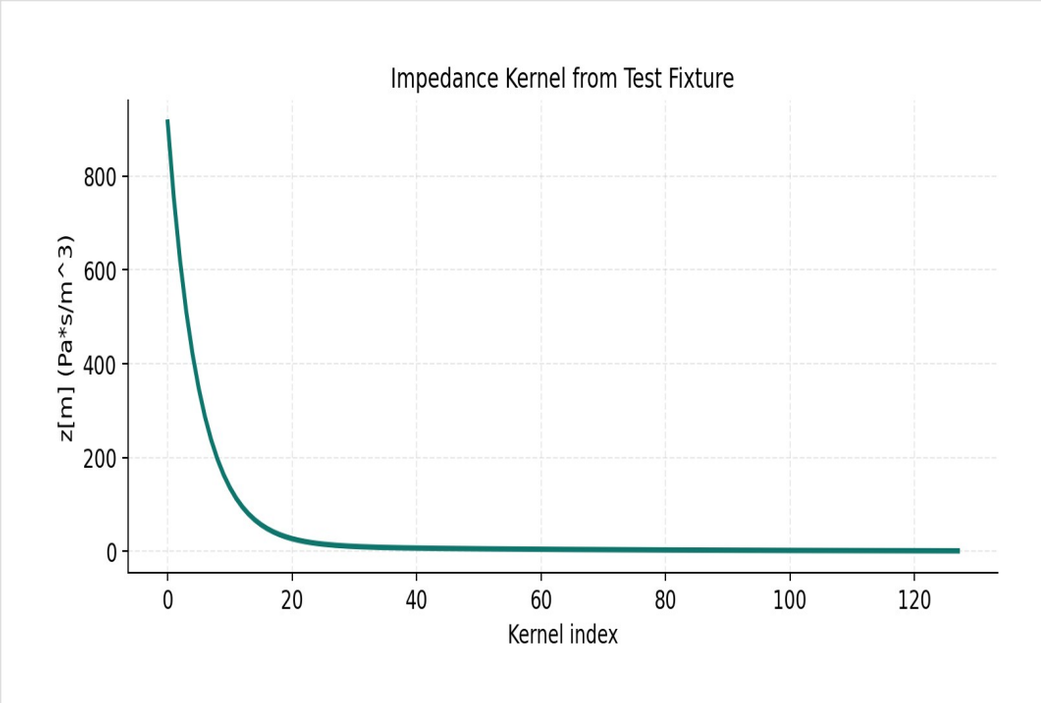
- Retry loops remain deterministic
- State rollback is explicit and controlled
- Accept/commit boundary is clearer

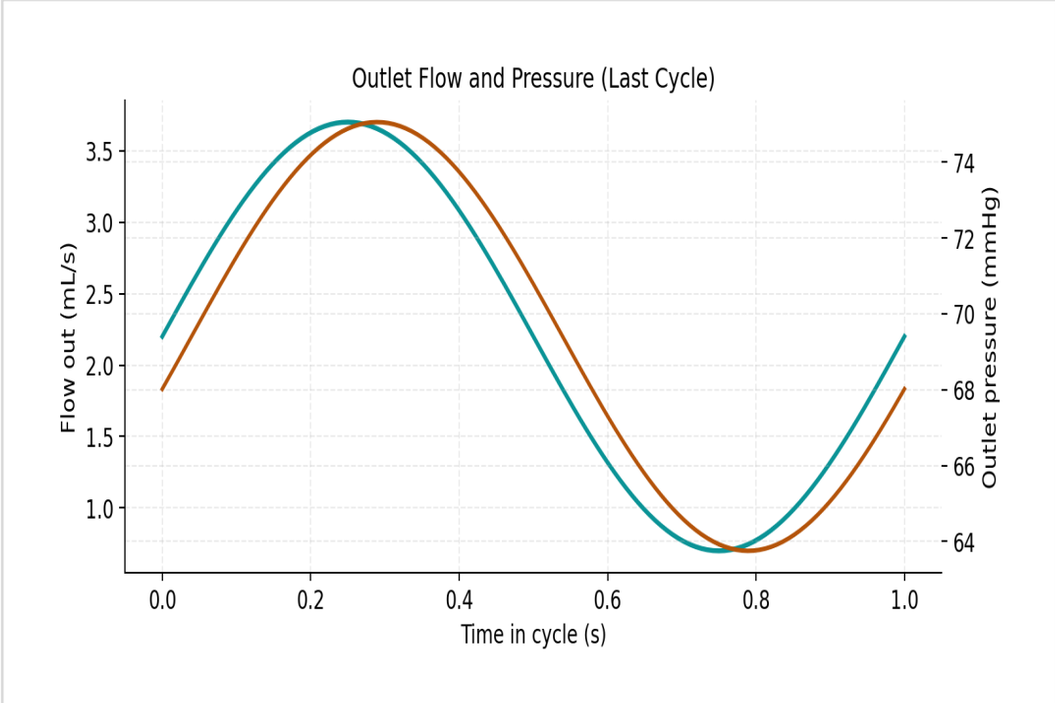
Full 0D

- Configuration remains straightforward
- Validation checks catch kernel and timestep inconsistencies
- Reproducibility improves through explicit persistent-state handling

Evidence and Takeaway

Tests and fixtures support the implementation claims and runtime behavior





- Impedance fixture validates kernel and waveform behavior
- Interface test_04 validates retry determinism and commit effect
- Overall result: targeted implementation with stable coupling behavior